

Application Python

Apprenant : Cheikh Ahmed Tidiane Ciss

Contexte

Vous venez d'intégrer une entreprise spécialisée dans l'Intelligence Artificielle.

Les Data Scientists manipulent quotidiennement des centaines de datasets (CSV et JSON). Avant de commencer les traitements avec Pandas, ils souhaitent disposer d'une application Python permettant de gérer ces jeux de données.

Votre mission consiste à développer progressivement une application console permettant de :

- gérer un catalogue de jeux de données
- enregistrer leurs caractéristiques
- effectuer des recherches
- afficher des statistiques
- sauvegarder les informations dans des fichiers
- charger automatiquement les données enregistrées
- gérer les erreurs d'utilisation.

1) **Créez** *datasetManager*, le dossier racine de l'application

2) **Créez** le fichier *main.py*, point d'entrée de l'application, pour contenir tout le code de la partie 1 à la partie 8.

Partie 1 : Types de base, variables, Entrées et sorties

3) Demandez à l'utilisateur de saisir les métadonnées d'un dataset :

```
PS C:\Users\cissc\OneDrive\Documents\datasetManager> & C:\Users\cissc\local\bin\python3.14.exe c:/Users/cissc/
cuments/datasetManager/datasetManager/main.py
Entrez le nom du dataset : ventes
Entrez le domaine du dataset : Santé; Agriculture; Environnement... etc : sante
Entrez le nombre de lignes du dataset : 200
Entrez le nombre de colonnes du dataset : 6
Entrez la taille du dataset en Mo : 21
Le dataset est-il public ? (True/False) : True
Choisissez le format du document (csv/json) : csv
```

4) Affichez ensuite un résumé formaté.

```
Résumé du dataset 'ventes':  
Domaine: sante  
Nombre de lignes: 200  
Nombre de colonnes: 6  
Taille: 21.0 Mo  
Public: True  
Format: csv  
PS C:\Users\cissc\OneDrive\Documents\datasetManager> |
```

Partie 2 – Structures de contrôle

5) Créez un menu interactif (provisoire)

```
PS C:\Users\cissc\OneDrive\Documents\datasetManager> & C:\Users\cissc\.local\bin\python3  
ents\datasetManager\datasetManager/main.py  
  
----- MENU -----  
1. Ajouter un dataset  
2. Afficher les datasets  
3. Rechercher  
4. Quitter  
Choisissez une option (1-4) : |
```

Partie 3 – Dictionnaires

6) Créez un dictionnaire pour stocker les métadonnées de chaque dataset

```
# Stockage des métadonnées dans un dictionnaire  
datasets[nom_dataset] = {  
    "domaine": domaine,  
    "nb_lignes": nb_lignes,  
    "nb_colonnes": nb_colonnes,  
    "taille_mo": taille_mo,  
    "public": public,  
    "format": format_doc  
}  
  
Résumé du dataset 'Titanic':  
Domaine: Transport  
Nombre de lignes: 891  
Nombre de colonnes: 12  
Taille: 48.0 Mo  
Public: True  
Format: csv  
PS C:\Users\cissc\OneDrive\Documents\datasetManager> |
```

Partie 4 – Tuples

7) Créez un tuple contenant les domaines autorisés.

("Santé", "Finance", "Agriculture", "Transport", "Education")

8) Vérifiez que le domaine saisi, à la question 3, appartient au tuple.

```
# 7) Créez un tuple contenant les domaines autorisés.  
domaines_autorises = ("Santé", "Finance", "Agriculture", "Transport", "Education")  
  
# 8) Vérifiez que le domaine saisi, à la question 3, appartient au tuple.  
if domaine not in domaines_autorises:  
    print("Erreur : Le domaine saisi n'est pas autorisé.")  
else:  
    print("Le domaine saisi est autorisé.") You, now * Uncommitted changes  
  
Résumé du dataset 'Titanic':  
Domaine: Transport  
Nombre de lignes: 891  
Nombre de colonnes: 12  
Taille: 48.0 Mo  
Public: True  
Format: csv  
Le domaine saisi est autorisé.  
PS C:\Users\cissc\OneDrive\Documents\datasetManager> |
```

Partie 5 – Listes

```
# Partie 5 - Listes  
# 9) Créez une liste contenant les datasets. Chaque ajout est enregistré dans la liste  
datasets_list = []  
for nom, metadonnees in datasets.items():  
    datasets_list.append(nom, metadonnees)  
  
# 10) Ajoutez les fonctionnalités : ajouter, trier, rechercher, modifier et supprimer  
# Ajouter un dataset  
def ajouter_dataset():  
    nom_dataset = input("Entrez le nom du dataset : ")  
    domaine = input("Entrez le domaine du dataset : (Santé, Finance, Agriculture, Transport et Education) : ")  
    nb_lignes = int(input("Entrez le nombre de lignes du dataset : "))  
    nb_colonnes = int(input("Entrez le nombre de colonnes du dataset : "))  
    taille_mo = float(input("Entrez la taille du dataset en Mo : "))  
    public = bool(input("Le dataset est-il public ? (True/False) : "))
```

Partie 6 – Compréhensions (Listes et Dictionnaires)

11) Affichez les statistiques comme par exemple :

```
# Partie 6 - Compréhensions (Listes et Dictionnaires)
# 11) Affichez les statistiques sur les datasets : nombre de datasets, nombre total de lignes, nombre moyens de colonnes, ...

print("\nStatistiques sur les datasets :")
print(f"Nombre de datasets : {len(datasets)}")
print(f"Nombre total de lignes : {sum(metadonnees['nb_lignes'] for metadonnees in datasets.values())}")
print(f"Nombre moyen de colonnes : {sum(metadonnees['nb_colonnes'] for metadonnees in datasets.values()) / len(datasets) if len(datasets) > 0 else 0}")
print(f"Datasets publics : {sum(1 for metadonnees in datasets.values() if metadonnees['public'])}")
print(f"Datasets privés : {len(datasets) - sum(1 for metadonnees in datasets.values() if metadonnees['public'])}")
print(f"Datasets au format CSV : {sum(1 for metadonnees in datasets.values() if metadonnees['format'] == 'csv')}")
print(f"Datasets au format JSON : {sum(1 for metadonnees in datasets.values() if metadonnees['format'] == 'json')}")

for domaine in domaines_autorises:
    count = sum(1 for metadonnees in datasets.values() if metadonnees['domaine'] == domaine)
    print(f"Répartition par domaine '{domaine}' : {count}")
```

```
Statistiques sur les datasets :
Nombre de datasets : 3
Nombre total de lignes : 8674
Nombre moyen de colonnes : 10.333333333333334
Datasets publics : 3
Datasets privés : 0
Datasets au format CSV : 2
Datasets au format JSON : 1
Répartition par domaine 'Santé' : 0
Répartition par domaine 'Finance' : 1
Répartition par domaine 'Agriculture' : 1
Répartition par domaine 'Transport' : 1
Répartition par domaine 'Education' : 0
PS C:\Users\ciscc\OneDrive\Documents\datasetManager>
```

Partie 7 – Fichiers

12) Créez le fichier datasets.csv pour :

- Sauvegarder les données

```
# Partie 7 - Les Fichiers :
# 12) Créez le fichier datasets.csv pour : sauvegarder les métadonnées des datasets, recharger et afficher les données.

with open("datasets.csv", "w") as f:
    f.write("nom,domaine,nb_lignes,nb_colonnes,taille_mo,public,format\n")
    for nom, metadonnees in datasets.items():
        f.write(f"{nom},{metadonnees['domaine']},{metadonnees['nb_lignes']},{metadonnees['nb_colonnes']},{metadonnees['taille_mo']},{metadonnees['public']},{metadonnees['format']}\n")
```

```
# Recharger les données depuis le fichier CSV
datasets_recharge = {}
with open("datasets.csv", "r") as f:
    next(f) # Ignorer l'en-tête
    for line in f:
        nom, domaine, nb_lignes, nb_colonnes, taille_mo, public, format_doc = line.strip().split(",")
        datasets_recharge[nom] = {
            "domaine": domaine,
            "nb_lignes": int(nb_lignes),
            "nb_colonnes": int(nb_colonnes),
            "taille_mo": float(taille_mo),
            "public": public == 'True',
            "format": format_doc
        }
```

- Recharger et afficher les données

```
# Afficher les données rechargées
print("\nDonnées rechargées depuis le fichier datasets.csv :")
for nom, metadonnees in datasets_recharge.items():
    print(f"- {nom}: {metadonnees}")
```

```
Données rechargées depuis le fichier datasets.csv :
- Titanic: {'domaine': 'Transport', 'nb_lignes': 891, 'nb_colonnes': 12, 'taille_mo': 48.0, 'public': True, 'format': 'csv'}
- Ventes: {'domaine': 'Finance', 'nb_lignes': 452, 'nb_colonnes': 10, 'taille_mo': 45.0, 'public': True, 'format': 'csv'}
- Recoltes: {'domaine': 'Agriculture', 'nb_lignes': 784, 'nb_colonnes': 5, 'taille_mo': 36.0, 'public': True, 'format': 'json'}
PS C:\Users\cissc\OneDrive\Documents\datasetManager>
```

Partie 8 – Exceptions

Plusieurs erreurs sont constatées :

- L'utilisateur saisit un texte au lieu d'un nombre
- Le fichier n'existe pas
- Le dataset recherché n'existe pas
- Le fichier est vide

13) Gérez ces exceptions pour que le programme continue de fonctionner.

```
# Partie 8 - Exceptions
# 13) Gérez les exceptions pour les entrées utilisateur et les opérations sur les fichiers.
def safe_input(prompt, expected_type):
    while True:
        try:
            value = input(prompt)
            if expected_type == int:
                return int(value)
            elif expected_type == float:
                return float(value)
            elif expected_type == bool:
                return value.lower() in ['true', '1', 'yes']
            else:
                return value
        except ValueError:
            print(f"Erreur : Veuillez entrer un {expected_type.__name__} valide.")
```

Partie 9 – Fonctions

14) Refactorisez le programme en créant les fonctions suivantes :

afficher_menu(), ajouter_dataset(), afficher_datasets(), supprimer_dataset(),
rechercher_dataset(), trier_dataset(), modifier_dataset(), sauvegarder(), recharger() et
statistiques()

```
# Partie 9 - Fonctions
# 14) Refactorisez le programme en créant les fonctions suivantes : afficher_menu(), ajouter_dataset(), trier_datasets()

def afficher_menu():
    print("\n----- MENU -----")
    print("1. Ajouter un dataset")
    print("2. Afficher les datasets")
    print("3. Rechercher")
    print("4. Modifier un dataset")
    print("5. Supprimer un dataset")
    print("6. Afficher les statistiques")
    print("7. Sauvegarder les données")
    print("8. Recharger les données")
    print("9. Quitter")

def ajouter_dataset():
    nom_dataset = safe_input("Entrez le nom du dataset : ", str)
    domain = safe_input("Entrez le domaine du dataset : Santé, Finance, Agriculture, Transport et Éducation : ", str)
    nb_lignes = safe_input("Entrez le nombre de lignes du dataset : ", int)
    nb_colonnes = safe_input("Entrez le nombre de colonnes du dataset : ", int)
```

```
# Appel des fonctions pour tester le programme

afficher_menu()
ajouter_dataset()
afficher_statistiques()
supprimer_dataset()
rechercher_dataset()
trier_datasets()
modifier_dataset()
recharger_donnees()
afficher_statistiques()
safe_input()
```

Partie 10 – Modules

15) Pour éviter des problèmes de maintenance, répartissez le code dans plusieurs modules.

